Computer Organization & Assembly Language

Lecture / Lab-005

Register Spilling in MIPS

Register Spilling in MIPS

- Register Spilling: Temporarily saving register values to memory (stack) so registers can be reused.
- Why? CPU registers are limited.
- $sp: Manages stack memory.
- sw / lw: Save and restore register values.
- Leaf & Non-Leaf: Register spilling can occur in both types of procedures

Quick Check: Leaf vs. Non-Leaf Function

- Leaf Function
  - Does NOT call another function
  - Does NOT need to save `$ra`
- Non-Leaf Function
  - Calls another function
  - MUST save and restore `$ra`

Quick Check: Stack Pointer ($sp)

- Holds address of top of stack
- Stack is memory used for temporary saves
- Stack grows downward (toward smaller addresses)
- Allocate space:
  - `addiu $sp, $sp, -4`
- Save value:
  - `sw $t0, 0($sp)`
- Restore value:
  - `lw $t1, 0($sp)`
- Deallocate space:
  - `addiu $sp, $sp, 4`

Stack Pointer ($sp)

- It holds the address of the top of the current stack frame, and it changes when you allocate/free stack space.

```
.text
.globl main
main:
  li  $a0, 17
  li  $a1, 5
  jal divmod

  # print quotient ($v0)
  move $a0, $v0
  li   $v0, 1
  syscall

  # exit
  li   $v0, 10
  syscall
```

```
# leaf helper that prints newline
print_newline:
  li   $a0, 10
  li   $v0, 11
  syscall
  jr   $ra


# divmod is NOT leaf anymore
#(it calls print_newline),
# so it must save/restore $ra.
divmod:
  addi $sp, $sp, -4
  sw   $ra, 0($sp)
```

```
div  $a0, $a1
  mflo $v0
  mfhi $v1

# jal writes the return address into $ra
  jal  print_newline

  lw   $ra, 0($sp)
  addi $sp, $sp, 4
  jr   $ra
```

A leaf function is a function that does not call any other function

Register Spilling Format

- Saving and Restoring a Register

```
addiu $sp, $sp, -4     # Allocate 4 bytes
sw    $t0, 0($sp)      # Save $t0 on stack

# Reuse $t0 for another calculation

lw    $t0, 0($sp)      # Restore original value
addiu $sp, $sp, 4      # Deallocate stack space
```

Key takeaway: Register spilling allows the CPU to reuse registers without permanently losing values that will be needed later.

Note that saving $ra in a non-leaf procedure preserves the return address; saving a temporary value to free a register for reuse demonstrates register spilling.

Saving $ra vs. Register Spilling

1. Saving $ra (Preserving Return Address)

```
sw $ra, 0($sp)
```

- Done in non-leaf procedures to preserve the return address.
- Necessary because another jal overwrites $ra.

2. Register Spilling

```
sw $t0, 0($sp)    # Save value
# Reuse $t0
lw $t0, 0($sp)    # Restore val
```

Saving $ra	Register Spilling
Preserves the return address	Frees a register for reuse
Supports correct function returns	Supports register allocation
Common in non-leaf procedures	Possible in leaf and non-leaf procedures

- Done to free a register for another calculation.
- Can occur in both leaf and non-leaf procedures.

Register Spilling

When many variables are alive at the same time, registers may not be sufficient. The compiler spills values to the stack.

```
# a in $a0, b in $a1, c in $a2, d in $a3

add   $t0, $a0, $a1      # x = a + b

addiu $sp, $sp, -4       # allocate space
sw    $t0, 0($sp)        # spill x

add   $t0, $a2, $a3      # y = c + d

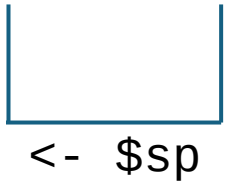
lw    $t1, 0($sp)        # restore x
addiu $sp, $sp, 4        # deallocate space

mul   $t2, $t1, $t0      # z = x * y
add   $v0, $t2, $t1      # return z + x
jr    $ra
```

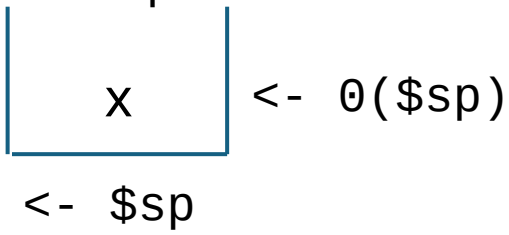
C++ code idea (too many “live” variables at once)

```
int f(int a, int b, int c, int d) {
  int x = a + b;
  int y = c + d;
  int z = x * y;
  return z + x;   // x is needed again later
}
```

Before spill:



After spill:



Register Spilling

- Given inputs `a = 2, b = 3, c = 4, d = 5,` write MIPS function f that returns `(a + b) (c + d) + (a + b)` and demonstrate register spilling by saving `x = a + b` on the stack, reusing a register, then reloading `x`.

```
#include <iostream>
using namespace std;

int f(int a, int b, int c, int d)
{
    int x = a + b;
    int y = c + d;
    int z = x * y;

    return z + x;
}

int main()
{
    int result = f(2, 3, 4, 5);

    cout << result << endl;

    return 0;
}
```

Requirement: demonstrate register spilling by
(1) computing x = a+b into a temp register,
(2) saving x on the stack using $sp,
(3) reusing that temp register to compute y = c+d,
(4) reloading x from the stack,
(5) finishing z = x*y and return z + x.

Register Spilling

- Occurs when registers are insufficient
- Compiler saves value to stack
- Later reloads value when needed
- Stack acts as temporary overflow storage

Given inputs `a = 2, b = 3, c = 4, d = 5,` write MIPS function f that returns `(a + b) (c + d) + (a + b)` and demonstrate register spilling by saving `x = a + b` on the stack, reusing a register, then reloading x.

```
x = a + b;
y = c + d;
z = x * y;
return z + x;
```

Requirement: demonstrate register spilling by
(1) computing x = a + b into a temp register,
(2) saving x on the stack using $sp,
(3) reusing that temp register to compute y = c + d,
(4) reloading x from the stack,
(5) finishing z = x * y and return z + x.

```
.data
newline: .asciiz "\n"
.text
.globl main

main:
    li $a0, 2
    li $a1, 3
    li $a2, 4
    li $a3, 5

    jal f
        #print value
        move $a0, $v0
        li $v0, 1
        syscall

    #print newline
    la $a0, newline
    li $v0, 4
    syscall

    #exit
    li $v0, 10
    syscall
```

```
f:
    add $t0, $a1, $a0

    addiu $sp, $sp, -4 #make space
    sw $t0, 0($sp) #save x


    #reuse $t0, for y = c + d
    add $t0, $a2, $a3

    #reload x (restore)
    lw $t1, 0($sp) #t1 =x
    addiu $sp, $sp, 4 #pop space

    #z = x * y
    mul $t2, $t1, $t0 #$t2 = x*y

    #return z + x
    add $v0, $t2, $t1 #$v0 = z +x
    jr $ra
```

In Class Task

Implement using MIPS Assembly using the test values:

$$a = 2, b = 3, c = 4, d = 5, e = 6, f = 7, g = 8, h = 9$$

Compute:

$$x = a + b = 2 + 3 = 5$$

$$y = c + d = 4 + 5 = 9$$

$$\mathrm{p} = \mathrm{x} \times y = 5 \times 9 = 45$$

$$u = e + f = 6 + 7 = 13$$

$$v = g + h = 8 + 9 + 17$$

$$\mathrm{q} = u \times v = 13 \times 17 = 221$$

Return:

$$p + q = 45 + 221 = 266$$

C++ CODE

```
int calculate(int a, int b, int c, int d,
              int e, int f, int g, int h)
{
    int x = a + b;
    int y = c + d;
    int p = x * y;

    int u = e + f;
    int v = g + h;
    int q = u * v;

    return p + q;
}
```

Non-Leaf Procedure and Register Spilling

Assume the following values are passed to a function f using standard MIPS calling conventions:
`a = 2, b = 3, c = 4, d = 5`
Write a MIPS procedure f that computes and returns:

`(a + b)(c + d) + (a + b)`

The procedure must call another procedure g to compute the multiplication.

The helper procedure g should compute:

`g(x,y) = x × y`

- Requirements
  - Use registers `$a0–$a3` for inputs and `$v0` for the return value.
  - Compute
    `x = a + b`
    `y = c + d`
  - Demonstrate register spilling by:
    saving x on the stack
    reusing a temporary register
    restoring `x` later from memory.
  - Since f calls another procedure, it must correctly save and restore `$ra`.
  - The final result returned in `$v0` should be: `(x × y) + x`

Expected Result
For the given inputs:
x = 5
y = 9
z = x*y = 45
return = z + x = 50

Non-Leaf Procedure and Register Spilling

C++ CODE:

```
#include <iostream>
using namespace std;

// Leaf function: performs multiplication
int g(int x, int y)
{
    return x * y;
}

// Non-leaf function: calls another function (g)
int f(int a, int b, int c, int d)
{
    int x = a + b;
    int y = c + d;

    int z = g(x, y);

    return z + x;
}

int main()
{
    int result = f(2, 3, 4, 5);

    cout << result << endl;

    return 0;
}
```

Non-Leaf Procedure and Register Spilling

MIPS CODE:

```
.data
newline: .asciiz "\n"
.text
.globl main
main:
    li $a0, 2
    li $a1, 3
    li $a2, 4
    li $a3, 5
    jal f

    # print result in $v0
    move $a0, $v0
    li   $v0, 1
    syscall

    # newline
    la   $a0, newline
    li   $v0, 4
    syscall

    # exit
    li $v0, 10
    syscall
```

```
f:
    addiu $sp, $sp, -8
    sw    $ra, 4($sp)        # save $ra

    add   $t0, $a0, $a1      # x = a + b
    sw    $t0, 0($sp)        # spill x

    add   $t0, $a2, $a3      # y = c + d (reuse $t0)

    lw    $a0, 0($sp)        # a0 = x
    move  $a1, $t0           # a1 = y
    jal   g                  # v0 = x*y

    lw    $t1, 0($sp)        # reload x
    add   $v0, $v0, $t1      # v0 = x*y + x

    lw    $ra, 4($sp)        # restore $ra
    addiu $sp, $sp, 8
    jr    $ra

# g(x,y) = x*y (leaf)
g:
    mul  $v0, $a0, $a1
    jr   $ra
```

In Class Task: Register Spilling in a Non-Leaf MIPS Procedure

Given C++ Program

Translate the following C++ program into MIPS assembly using QtSPIM.

```
#include <iostream>
using namespace std;
// Leaf function
int multiply(int x, int y)
{
    return x * y;
}
// Non-leaf function
int calculate(int a, int b, int c, int d)
{
    int x = a + b;
    int y = c + d;

    int z = multiply(x, y);

    return z - x;
}

int main()
{
    int result = calculate(3, 5, 4, 6);

    cout << result << endl;
    return 0;
}
```

In Class Task: Register Spilling in a Non-Leaf MIPS Procedure

equirements

our MIPS program must:

Pass arguments: Use $a0–$a3 to pass the four arguments to calculate.

Compute x: Calculate x = a + b using a temporary register.

Demonstrate register spilling: Save x on the stack using $sp and sw, then reuse the same temporary register to calculate y = c + d.

Implement a non-leaf procedure: calculate must call multiply using jal and correctly preserve its return address (

Implement a leaf procedure: multiply must multiply its two arguments and return the result in $v0.

Restore saved values: Reload x from the stack using lw after multiply returns, and restore $ra and $sp before returning to main.

Print the result: Display the final result in the QtSPIM console.

Expected Output

Given:

$a = 3, \quad b = 5, \quad c = 4, \quad d = 6$

The calculations are:

$$x = a + b = 3 + 5 = 8$$

$$y = c + d = 4 + 6 = 10$$

$$z = x \times y = 8 \times 10 = 80$$

$$\text{result} = z - x = 80 - 8 = \boxed{72}$$